

Supplementary Material
**Reinforced Diffusion on Graphs for
Decentralized Multi-Robot Navigation with
Smooth Formation Adaptation**

Author Names Omitted for Anonymous Review. Paper-ID [1919]

CONTENTS

I	Time Complexity Analysis	2
I-A	Time Complexity Analysis and Scalability of ReDiG	2
I-B	Time Complexity Analysis of Baseline Methods	2
II	Implementation Details	3
II-A	Model Architecture	3
II-B	Autonomy Stack Architecture	3
II-C	Hyperparameters	4
III	Training Details	4
III-A	Reward Design	4
IV	Experimental Details	5
IV-A	Runtime and Control Frequency	5
IV-B	Examples of Computing the CFI Evaluation Metric	5
V	Additional Experimental Results	7
	References	8

Algorithm 1: ReDiG Training

Input : Team graph $\mathcal{G}$, robot states $\{s_i\}$, replay buffer $\mathcal{M}$
Output : Trained graph network ϕ , denoising network ϵ_θ , and critic networks $Q_{\omega_1}, Q_{\omega_2}$

```
1 Initialize  $\phi, \epsilon_\theta, Q_{\omega_1}, Q_{\omega_2}$ ;  
2 while Not converged do  
3   Compute team-level embeddings  $\mathbf{h}_i = \phi(s_i, \mathcal{G})$ ;  
4   Sample actions  $\{\mathbf{a}_i\}^n \sim \{\pi_\theta(\mathbf{a}_i | \mathbf{h}_i)\}^n$ ;  
5   Compute  $\mathbb{E}_{\mathbf{h}_i, \mathbf{a}_i^0} [\|\epsilon_k - \epsilon_\theta(\mathbf{a}_i^0 + \epsilon_k, k | \mathbf{h}_i)\|_2^2]$ ;  
6   Update  $\epsilon_\theta$  based on  $\nabla_\theta \mathbb{E}_{\mathbf{h}_i, \mathbf{a}_i^0} [\cdot]$ ;  
7   Execute  $\{\mathbf{a}_i\}^n$  and insert transition  $(s_i, \mathbf{a}_i, R_i, s'_i)$  to  $\mathcal{M}$ ;  
8   Sample  $(s_i, \mathbf{a}_i, R_i, s'_i)$  from  $\mathcal{M}$ ;  
9   Compute  $\mathbb{E}_{\mathbf{h}_i, \mathbf{a}_i} [(Q_{\omega_1}(\mathbf{h}_i, \mathbf{a}_i) - y_i)^2 + (Q_{\omega_2}(\mathbf{h}_i, \mathbf{a}_i) - y_i)^2]$ ;  
10  Refine action  $\mathbf{a}_i = \mathbf{a}_i + \eta \nabla_{\mathbf{a}_i} Q_\omega(\mathbf{h}_i, \mathbf{a}_i)$  and insert updated transition  $(s_i, \mathbf{a}_i, R_i, s'_i)$  to  $\mathcal{M}$ ;  
11  Update  $Q_{\omega_1}$  and  $Q_{\omega_2}$  based on  $\nabla_{\omega_1} \mathbb{E}_{\mathbf{h}_i, \mathbf{a}_i} [\cdot]$  and  $\nabla_{\omega_2} \mathbb{E}_{\mathbf{h}_i, \mathbf{a}_i} [\cdot]$ ;  
12  Update  $\phi$  based on  $\nabla_\theta \mathbb{E}_{\mathbf{h}_i, \mathbf{a}_i^0} [\cdot]$ ,  $\nabla_{\omega_1} \mathbb{E}_{\mathbf{h}_i, \mathbf{a}_i} [\cdot]$ , and  $\nabla_{\omega_2} \mathbb{E}_{\mathbf{h}_i, \mathbf{a}_i} [\cdot]$ ;  
13 end  
14 return  $\phi, \epsilon_\theta, Q_{\omega_1}, Q_{\omega_2}$ 
```

I. TIME COMPLEXITY ANALYSIS

A. Time Complexity Analysis and Scalability of ReDiG

Training time complexity is dominated by $O(n^2)$ using Algorithm 1, where n is the number of robots. The decentralized graph network generates state embeddings has an $O(L_g n^2)$ complexity, where L_g is the number of GNN layers, n^2 is the number of edges computed for communications. The denoising network training has an $O(T_d B_d L_d)$ complexity, where T_d is the number of denoising steps, B_d is the sampled batch size, and L_d is the number of denoising network layers. The critic network training has an $O(B_c L_c n)$ complexity, where B_c is the sampled batch size, L_c is the number of critic network layers. The action refinement using gradient ascent has an $O(G_a B_a n)$ complexity, where G_a is the number of gradient steps, B_a is the sampled batch size, and each gradient step requires critic evaluation with $O(n)$ complexity. Combining all terms, the overall complexity for training is $O(I(n^2 + T_d B_d L_d + B_c L_c n + G_a B_a n))$, where I is the number of training iterations. *Execution time complexity* is dominated by $O(n)$. The complexities of embedding generation from graph and actions generation from diffusion policy are $O(L_g n)$ and $O(T_d L_d)$, the overall execution time complexity is $O(n + T_d)$.

The above time complexity provides a direct analytical measure of the scalability of our approach. Training involves quadratic interactions $O(n^2)$ due to pairwise message passing in the graph encoder, but this cost is amortized through batch parallelization, making it tractable even for large teams. During decentralized execution, the complexity reduces to $O(n + T_d)$, where the linear term reflects local neighbor aggregation and the constant term corresponds to the diffusion process. This separation of costs highlights that the per-robot computation remains bounded, and with decentralized parallelization, the effective execution complexity is dominated by the diffusion steps rather than the team size. Thus, the complexity analysis demonstrates that ReDiG is scalable and readily generalizes to larger robot teams in a fully decentralized manner.

B. Time Complexity Analysis of Baseline Methods

We also analyze the time complexity of baseline methods for comparison. While ReDiG exhibits a similar training complexity to these baselines, it achieves a lower execution complexity, enabling more efficient and scalable real-time performance.

Leader-Follower [1]. *Time complexity* is dominated by $O(n^2)$, where n is the number of robots. The leader's path planning has a complexity of $O(P)$, where P depends on the path planning algorithm used (e.g., A* or RRT). For A*, the complexity is typically $O(b^d)$, where b is the branching factor and d is the depth of the optimal solution. For RRT, it is typically $O(k \log k)$, where k is the number of sampled nodes. Each follower computes its position relative to the leader and potentially other followers to maintain formation, resulting in $O(n - 1)$ complexity per follower. For $n - 1$ followers, the total coordination complexity is $O((n - 1)^2) = O(n^2)$. Therefore, the overall time complexity is $O(P + n^2)$.

Online Reinforcement Learning with PPO [2]–[4]. *Training time complexity* is dominated by $O(n^2)$, where n is the number of robots. The GNN has an $O(L_m T_p n^2)$ complexity, where L_m is the number of layers in the GNN and T_p is the number of training iterations using Proximal Policy Optimization (PPO). The control network training has an $O(T_p (B n^2 + I B n))$ complexity, where B is the number of PPO rollouts to interact with the environment in each iteration, and I is the number of PPO training epochs. $O(B n^2)$ accounts for computing the advantage function, and $O(I B n)$ for updating the policy. Combining all terms, the overall complexity for training is $O(H L_h T_h D n^2 + L_m T_p n^2 + T_p (B n^2 + I B n))$. *Execution time complexity* is dominated by $O(n^2)$. The complexities of the GNN and control networks are $O(L_m n^2)$ and $O(n)$, respectively.

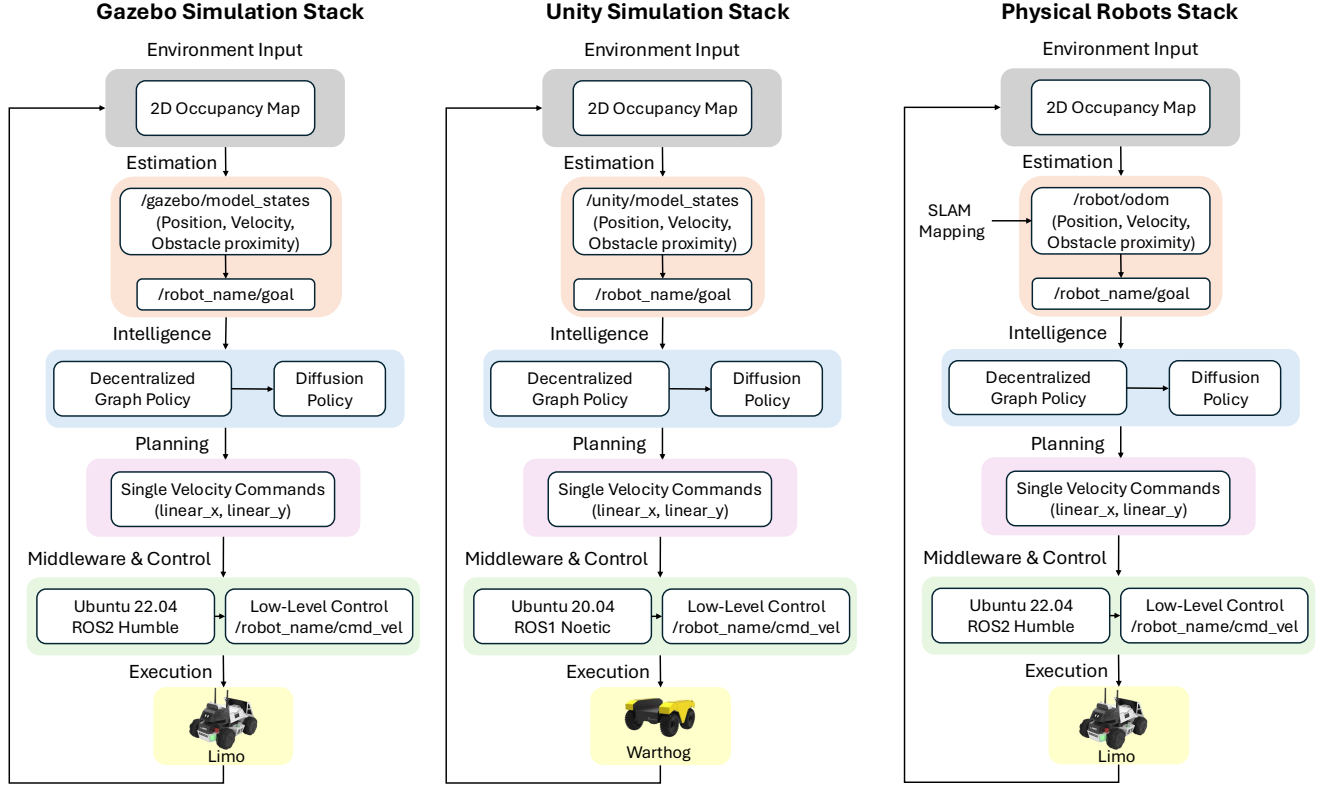


Fig. 1. Autonomy stacks for robotics simulations and physical robots.

II. IMPLEMENTATION DETAILS

A. Model Architecture

To implement ReDiG, we construct the robot team graph by connecting neighboring robots that are within a 2.0-meter spatial radius. The graph learning consists of an encoder with a weight matrix $\mathbf{W}^z$ of dimension 7×64 , followed by a single GNN layer with a weight matrix $\mathbf{W}^h$ of dimension 64×64 . The diffusion model employs a denoising network ϵ_θ set to a dimension of 40×256 with Mish activation functions. The critic includes two separate but identical value networks, Q_1 and Q_2 , both set to the dimension of 8×256 . In the spring-damper model of ReDiG, the hyper-parameter $\rho = 0.5$ is used to balance the contributions of the spring and damper forces. We generate synthetic data to train our ReDiG approach by randomly generating robot positions within a given team formation.

B. Autonomy Stack Architecture

We design autonomy stacks for three different platforms, including Gazebo simulation, Unity simulation, and physical robots, as illustrated in Figure 1. In all cases, each robot is assigned a unique namespace to support decentralized execution, which ensures that every robot runs its own autonomy stack, independently processing its local state and executing actions.

In Gazebo simulation, we use Ubuntu 22.04 with ROS2 Humble. The simulator provides a 2D occupancy map together with ground-truth robot states, including positions, velocities, and obstacle proximity. Goal positions are specified for each robot under its namespace. At every timestep, ReDiG generates velocity commands in the form of linear values, which are then executed by Limo robots through the `/cmd_vel` topic, where the linear command is translated into both linear and angular velocities for differential drive control.

Unity simulations are run on Ubuntu 20.04 with ROS1 Noetic. The Unity engine supplies the 2D occupancy map, robot poses, and obstacle proximity. After ReDiG computes velocity commands, Warthog robots execute them through the `/cmd_vel` topic under their respective namespaces. This stack provides a high-fidelity testing environment that replicates large-scale outdoor settings with accurate robot dynamics.

For physical robots, we use Ubuntu 22.04 with ROS2 Humble. Robot states are estimated through SLAM-based methods. Each Limo robot receives its state and goal under a dedicated namespace. ReDiG outputs single-step velocity commands that are executed in a decentralized manner through the `/cmd_vel` topic.

TABLE I
HYPERPARAMETERS USED FOR TRAINING ReDiG

Hyperparameter	Value	Hyperparameter	Value
Hidden layers	3	Hidden units per layer	256
Policy/value activation	Mish	Batch size	256
Discount factor γ	0.99	Target update rate	0.005
Policy learning rate	3×10^{-4}	Critic learning rate	3×10^{-4}
GNN learning rate	3×10^{-4}	Gradient clipping norm	2.0
Replay buffer size	1×10^6	Reward scaling factor	0.01
Diffusion steps K	100	β schedule	Cosine
Noise scale	1.0	GNN message dimension	32
GNN communication range	2.0	GNN activation	ReLU



(a) AgileX Limo Robots



(b) ClearPath Warthog Robots

Fig. 2. Robotic platforms used in our experiments. Figure 2(a) shows differential-drive ground robots AgileX Limo robots that are used in our real-world experiment. Figure 2(b) Clearpath Warthog robots are large, rugged, all-terrain differential-drive UGVs that are used in our Unity simulations.

C. Hyperparameters

All components of ReDiG, including the graph neural network, diffusion denoising network, and critic networks, are trained jointly for 30,000 epochs. Table I summarizes the complete set of hyperparameters, which are fixed and used consistently across all experiments. All training procedures are conducted on a machine equipped with a 16-core Intel i9 CPU, 32 GB RAM, and an NVIDIA RTX 4090 GPU.

III. TRAINING DETAILS

A. Reward Design

We present a reward design based on a spring-damper model, inspired by [3], [5], to enable adaptive formation control. The spring-damper reward is designed as follows:

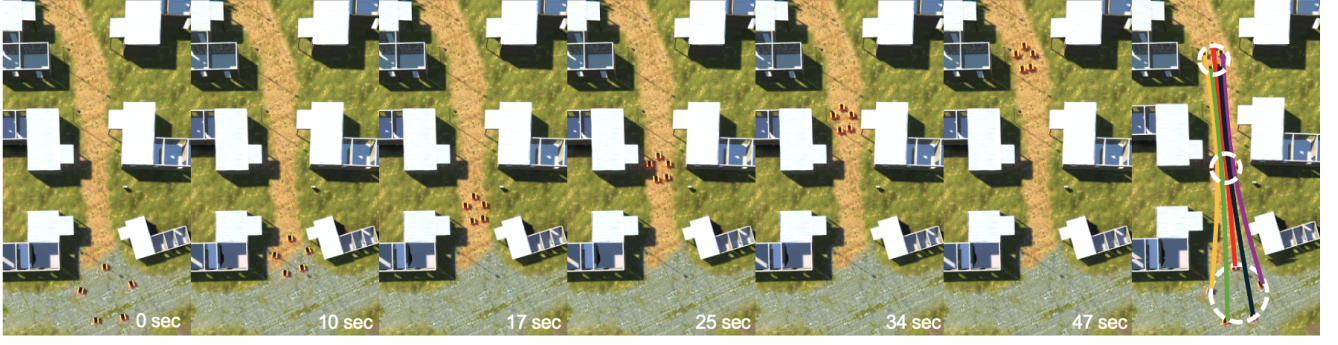
$$R^{adp} = \sum_{(i,j) \in \mathcal{V}} \left(-\rho |d_{i,j} - p_{i,j}| - (1 - \rho) q_{i,j} \right), \quad (1)$$

where $d_{i,j}$ denotes the desired inter-robot distance, $p_{i,j}$ is the actual distance between robots i and j , and $q_{i,j}$ represents their relative velocity magnitude. The hyperparameter $\rho \in [0, 1]$ balances the spring term, which penalizes deviations from the desired formation geometry, and the damper term, which suppresses relative motion to reduce oscillations.

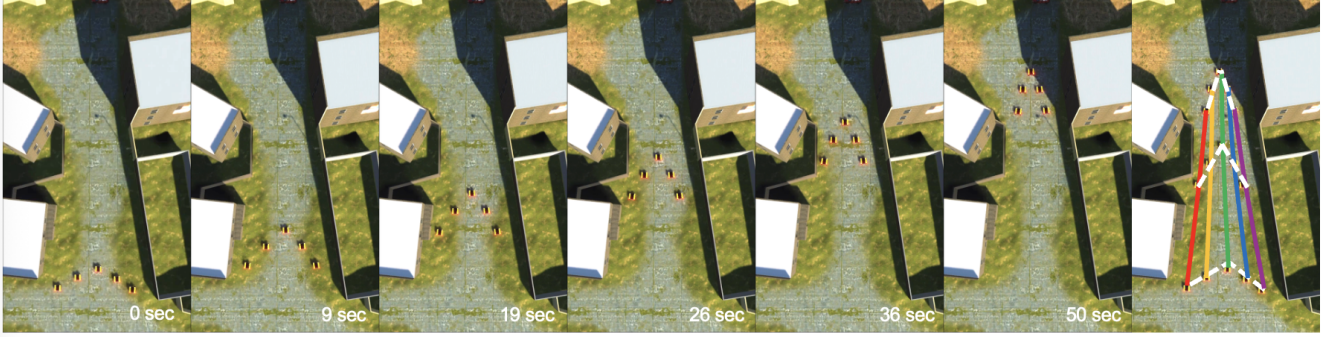
This spring-damper reward is combined with standard navigation and collision-avoidance rewards during training. Specifically, the navigation reward encourages progress toward the goal while penalizing collisions, and is defined as follows:

$$R_i^{nav} = \begin{cases} \mathbf{q}_i^\top \frac{\vec{r}_{\text{goal}}}{\|\vec{r}_{\text{goal}}\|} \|\mathbf{q}_i\|, & \text{if } \|\vec{r}_{\text{goal}}\| > 0, \\ 15.0, & \text{if } \|\mathbf{p}_i - \mathbf{g}_i\|_2 < 0.1, \\ -1.5, & \text{if } \|\mathbf{p}_i - \mathbf{p}_j\|_2 < 0.2, \end{cases} \quad (2)$$

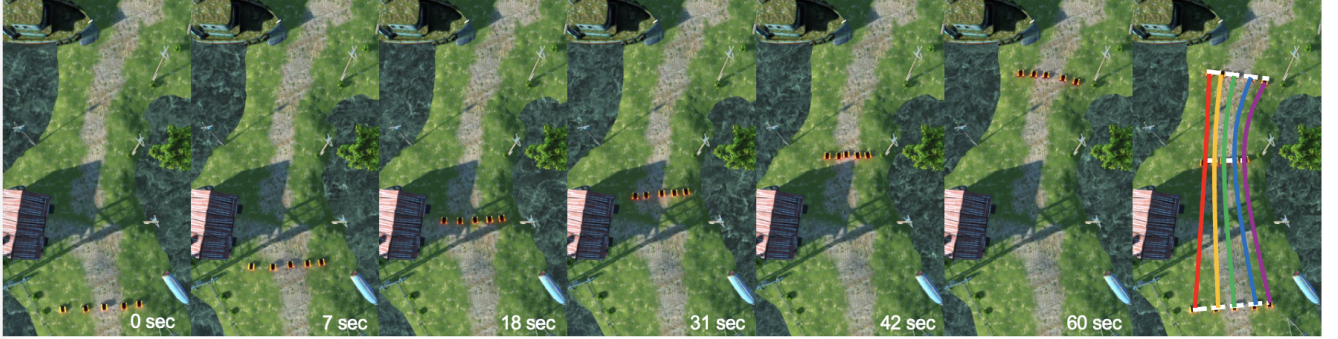
where $\vec{r}_{\text{goal}} = \mathbf{g}_i - \mathbf{p}_i$ is the vector pointing from the robot's current position to its goal. The first term rewards velocity alignment with the goal direction, scaled by the speed magnitude to encourage efficient forward motion. The second term provides a terminal reward for reaching the goal, while the third term penalizes collisions with other robots or obstacles. Together, these reward components enable ReDiG to jointly achieve goal-directed navigation, collision avoidance, and adaptive formation control.



(a) A team of five robots in a circle formation navigates through multiple narrow passages.



(b) A team of five robots in a wedge formation traverses a progressively narrowing corridor between buildings.



(c) A team of five Warthog robots with a line formation navigates through a narrow corridor on uneven terrain.

Fig. 3. Qualitative results on ReDiG for decentralized coordinated multi-robot navigation using a high-fidelity Unity3D simulations in ROS1. The experiments utilize differential-drive Warthog robots that maintain circle, wedge and line formations while traversing an unstructured outdoor field environment.

IV. EXPERIMENTAL DETAILS

A. Runtime and Control Frequency

ReDiG generates only the current velocity vector at each timestep for each individual robot, with one full diffusion pass corresponding to a single control action. This design eliminates the need to generate full trajectories, significantly reducing latency. In simulation, we employ a workstation equipped with an NVIDIA RTX 4090 GPU and run the full diffusion process with 100 denoising steps, achieving a control frequency of approximately 128.6 Hz during rollouts.

We also observe that using early stopping (e.g., 60 steps) can increase the frequency to around 223.0 Hz without compromising performance. For real-world deployment, both the diffusion model and the GNN are executed onboard each LIMO robot using an Intel NCU i7 processor. With early stopping, higher control frequencies can be achieved when required, ensuring real-time operation under hardware constraints.

B. Examples of Computing the CFI Evaluation Metric

To evaluate adaptive formation adaptation, we use the Contextual Formation Integrity (CFI) metric [3], defined as

$$\text{CFI} = w \left(1 - \delta^{-1} \min(|r - (\xi + \delta)|, |r - (\xi - \delta)|) \right) + (1 - w)\tau, \quad (3)$$



(a) A team of five physical robots with a circular formation navigates through a narrow doorway in a hallway.



(b) A team of five robots with a wedge formation navigates through a narrow exit.



(c) A team of four robots in a line formation navigates through a narrow door.

Fig. 4. Qualitative results on smooth formation adaptation are demonstrated across diverse indoor environments, with varying formation shapes and different team sizes. Using multiple differential-drive Limo robots, the teams successfully maintain circle, wedge, and line formations under a range of indoor scenarios.

where the first term evaluates the efficiency of the robot team in utilizing the available corridor gap. Here, r is the maximum radius of the robot team, ξ denotes the corridor-width threshold with a safety margin, and δ represents an uncertainty tolerance, with smaller values imposing stricter formation requirements. The second term, $\tau \in [0, 1]$, evaluates the integrity of the team formation. CFI therefore jointly evaluates how effectively the robot team utilizes the available space and maintains the desired formation, with the trade-off controlled by w . Higher CFI values indicate better performance. In our experiments, we set $w = 0.5$ to weight gap utilization and formation integrity equally and set δ to twice the width of the robot used in the corresponding experiments.

For a team of n robots, the formation-integrity term τ is computed according to the desired formation. We denote clipping to $[0, 1]$ by $[x]_0^1 = \min(1, \max(0, x))$. Specifically:

- **Circle formation:**

$$\tau = \left[1 - \frac{1}{n} \sum_{i=1}^n \frac{|\theta_i - \theta^*|}{\theta^*} \right]_0^1, \quad \theta^* = \frac{(n-2)180^\circ}{n}, \quad (4)$$

where θ_i is the interior angle associated with the i -th robot and θ^* is the ideal interior angle of a regular n -sided polygon, which approximates a circle as the number of robots increases.

- **Wedge formation:**

$$\tau = \left[1 - \frac{2|H_l - H_r|}{H_l + H_r} - \frac{|2H_m - H_b|}{H_b} \right]_0^1, \quad (5)$$

where H_l , H_r , H_m , and H_b denote the lengths of the left, right, middle, and base segments of the isosceles-triangle geometry formed by the robots.

- **Line formation:**

$$\tau = \left[1 - \frac{1}{n-1} \sum_{i=1}^{n-1} \frac{H_{i,i+1}}{L} \right]_0^1, \quad (6)$$

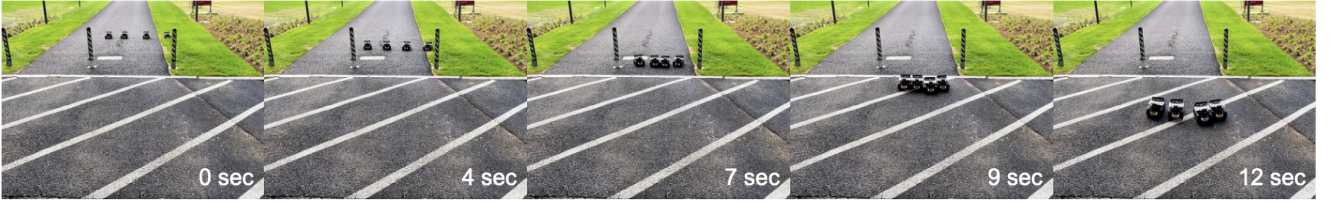
where $H_{i,i+1}$ denotes the deviation between neighboring robots i and $i+1$, and L denotes the full width of the robot team.



(a) A team of five physical robots with a circular formation navigates through a narrow passage between two concrete security bollards.



(b) A team of five robots with a wedge formation navigates through a forest-like environment characterized by narrow corridors, scattered trees, and surrounding obstacles.



(c) A team of four robots in a line formation navigates through a narrow pathway, where robots access is restricted by two sticks marking the boundaries.

Fig. 5. Qualitative results of ReDiG demonstrate decentralized coordinated navigation with smooth adaptive formation control, using different numbers of differential-drive Limo robots that maintain different formations across various unstructured outdoor environments.

V. ADDITIONAL EXPERIMENTAL RESULTS

We present comprehensive qualitative experimental results to demonstrate the applicability and generalizability of ReDiG. Evaluations are conducted across diverse environments, including Gazebo with ROS2, a high-fidelity 3D Unity simulator with ROS1, and physical multi-robot teams with ROS2 in both indoor and outdoor settings.

Figure 2 illustrates the robotic platforms used in our experiments. In simulation, we employ Clearpath Warthog robots with differential-drive kinematics, while real-world experiments are conducted using Agilex LIMO robots, also operating under differential-drive control. Since ReDiG directly outputs linear velocity commands for each individual robot, the proposed method is agnostic to specific hardware platforms and can be readily deployed on any velocity-controlled ground robot across both simulated and real-world environments.

We first present additional qualitative results obtained in Unity-based simulations, as shown in Figure 3. The Unity environment models outdoor field scenarios featuring narrow passages formed by buildings or constrained by flooded ground regions. These settings pose significant challenges. First, the simulated Clearpath Warthog robots successfully convert the linear velocity commands generated by ReDiG into wheel velocities, allowing them to accurately track the planned trajectories. Second, the outdoor environments contain multiple narrow passages that are chained together into long and complex routes, requiring the robot team to continuously adapt its formation while navigating toward the goal smoothly to avoid excessive jerk caused by frequent formation adjustments. ReDiG overcomes these challenges and enable smooth formation adaptation across different scenarios. Figure 5(a) illustrates a scenario consists of multiple consecutive passages with consistent narrow space. ReDiG enables the robot team in a circle formation to continuously adapt the formation into a smaller-radius circle and maintain this compact configuration throughout the entire navigation process. Figure 5(b) illustrates a scenario in which the available corridor width gradually decreases along the navigation path. The robot team initially adopts a wide wedge formation and incrementally adapts its configuration in response to the progressively narrowing corridor, ultimately forming a compact wedge at the end of the navigation. Figure 5(c) depicts a team of robots traversing uneven terrain in a line formation. ReDiG enables the team to temporarily contract into a shorter line configuration to pass through the constrained region, and then gradually expand back to its original formation size once sufficient free space is restored (e.g., at 60,s). Notably, this formation adaptation is achieved through smooth and coordinated motion, resulting in efficient and safe team navigation.

We next present qualitative results from real-world experiments conducted with teams of physical Agilex LIMO robots in both indoor and outdoor environments. Each robot runs ROS2 and communicates with other team members via Wi-Fi-based broadcast messaging. Figure 4 shows representative indoor scenarios involving constrained spaces, such as narrow doorways.

ReDiG enables a team of five robots in both circle and wedge formations to successfully traverse these narrow passages by continuously adapting their formation shape. In addition, a team of four robots operating in a line formation is able to pass through an extremely narrow doorway. Figure 5 presents real-world outdoor experiments conducted on unstructured terrain. The scenarios include navigation through a narrow passage between two concrete bollards, traversal of obstacle-constrained regions formed by scattered trees in a forest-like environment, and movement along a narrow pathway defined by two sticks marking the boundaries. These results indicate the strong adaptability of the robot team to unseen environments. ReDiG enables robust performance on uneven terrain, where wheel slippage introduces action uncertainty. Despite these challenges, ReDiG allows the robot team to navigate smoothly and stably while adapting its formation online, further validating the effectiveness of the proposed approach.

REFERENCES

- [1] B. Reily, C. Reardon, and H. Zhang, "Leading multi-agent teams to multiple goals while maintaining communication," in *Robotics Science and Systems*, 2020.
- [2] J. Blumenkamp, S. Morad, J. Gielis, Q. Li, and A. Prorok, "A framework for real-world multi-robot systems running decentralized GNN-based policies," in *International Conference on Robotics and Automation*, 2022.
- [3] Z. Deng, P. Gao, W. J. Jose, C. Reardon, M. Wigness, J. Rogers, and H. Zhang, "Coordinated multi-robot navigation with formation adaptation," in *IEEE International Conference on Robotics and Automation*, 2025.
- [4] Z. Deng, P. Gao, W. J. Jose, M. Wigness, J. G. Rogers III, B. Reily, C. M. Reardon, and H. Zhang, "Subteaming and adaptive formation control for coordinated multi-robot navigation," in *Conference on Robot Learning*, 2025.
- [5] C. Gabellieri, A. Palleschi, and L. Pallottino, "Force-based formation control of omnidirectional ground vehicles," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2021.